LeetCode 热题 HOT 100 完整题解

> 本文档包含LeetCode HOT 100全部100道题的完整题目描述与详细解题思路
>
> 整理时间：2026年3月

目录

1. 哈希
2. 双指针
3. 滑动窗口
4. 普通数组
5. 矩阵
6. 链表
7. 二叉树
8. 图论
9. 回溯
10. 二分查找
11. 栈
12. 堆
13. 贪心算法
14. 动态规划
15. 多维动态规划
16. 技巧

一、哈希

1. 两数之和 (Two Sum)

题号：1
难度：简单
链接：https://leetcode.cn/problems/two-sum/

题目描述： 给定一个整数数组 nums 和一个整数目标值 target，请你在该数组中找出和为目标值 target 的那两个整数，并返回它们的数组下标。

你可以假设每种输入只会对应一个答案。但是，数组中同一个元素在答案里不能重复出现。

你可以按任意顺序返回答案。

示例：

```
输入：nums = [2,7,11,15], target = 9
输出：[0,1]
解释：因为 nums[0] + nums[1] == 9，返回 [0, 1]。
```

解题思路：

方法一：暴力枚举（不推荐） - 思路：遍历数组，对于每个元素，再遍历其后面的所有元素，看两数之和是否等于target - 时间复杂度：$O(n^2)$ - 空间复杂度：O(1)

方法二：哈希表（推荐） - 思路：使用哈希表存储已经遍历过的数字及其下标。对于当前数字x，在哈希表中查找target-x是否存在 - 关键：先查后存，避免同一个元素使用两次 - 时间复杂度：O(n)，只需要遍历一次数组 - 空间复杂度：O(n)，哈希表存储n个元素

核心代码逻辑：

```
创建空哈希表map
遍历数组，对于每个元素num及其下标i：
    complement = target - num
    如果complement在map中：
        返回[map[complement], i]
    将num和i存入map：map[num] = i
```

---

2. 字母异位词分组 (Group Anagrams)

题号：49
难度：中等
链接：https://leetcode.cn/problems/group-anagrams/

题目描述： 给你一个字符串数组，请你将字母异位词组合在一起。可以按任意顺序返回结果列表。

字母异位词是由重新排列源单词的所有字母得到的一个新单词。

示例：

```
输入: strs = ["eat", "tea", "tan", "ate", "nat", "bat"]
输出: [["bat"],["nat","tan"],["ate","eat","tea"]]
```

解题思路：

核心思想：字母异位词排序后得到的字符串相同

方法一：排序作为键 - 思路：将每个字符串排序，排序后的字符串作为哈希表的键，原字符串加入对应的值列表 - 时间复杂度：O(n × k log k)，n是字符串个数，k是字符串最大长度 - 空间复杂度：O(n × k)

方法二：字符计数作为键 - 思路：使用26个长度的数组统计每个字符出现次数，将计数数组转为字符串作为键 - 时间复杂度：O(n × k) - 空间复杂度：O(n × k) - 优势：避免排序，时间复杂度更优

---

3. 最长连续序列 (Longest Consecutive Sequence)

题号：128
难度：中等
链接：https://leetcode.cn/problems/longest-consecutive-sequence/

题目描述： 给定一个未排序的整数数组 nums ，找出数字连续的最长序列（不要求序列元素在原数组中连续）的长度。

请你设计并实现时间复杂度为 O(n) 的算法解决此问题。

示例：

```
输入：nums = [100,4,200,1,3,2]
输出：4
解释：最长数字连续序列是 [1, 2, 3, 4]。它的长度为 4。
```

解题思路：

核心思想：只从序列的起点开始计数

步骤： 1. 将所有数字存入哈希集合，实现O(1)查找 2. 遍历集合中的每个数字num： - 如果num-1不在集合中，说明num是某个序列的起点 - 从num开始，不断查找num+1, num+2…，直到不连续 - 记录当前序列长度，更新最大值

关键点： - 通过判断num-1是否存在，确保每个序列只被计算一次 - 避免了重复计算，保证O(n)时间复杂度

时间复杂度：O(n)，每个数字最多被访问两次（一次作为起点判断，一次在序列中）
空间复杂度：O(n)

---

4. 只出现一次的数字 (Single Number)

题号：136
难度：简单
链接：https://leetcode.cn/problems/single-number/

题目描述： 给定一个非空整数数组，除了某个元素只出现一次以外，其余每个元素均出现两次。找出那个只出现了一次的元素。

说明： 你的算法应该具有线性时间复杂度。 你可以不使用额外空间来实现吗？

示例：

```
输入: [2,2,1]
输出: 1
```

解题思路：

核心思想：利用异或运算的性质

异或运算性质： - a ^ a = 0（任何数与自身异或为0） - a ^ 0 = a（任何数与0异或为其本身） - 异或运算满足交换律和结合律

算法： - 将所有数字进行异或运算 - 成对出现的数字异或结果为0 - 最后剩下的就是只出现一次的数字

时间复杂度：O(n)
空间复杂度：O(1)，满足题目要求

二、双指针

5. 移动零 (Move Zeroes)

题号：283
难度：简单
链接：https://leetcode.cn/problems/move-zeroes/

题目描述： 给定一个数组 nums，编写一个函数将所有 0 移动到数组的末尾，同时保持非零元素的相对顺序。

请注意 ，必须在不复制数组的情况下原地对数组进行操作。

示例：

```
输入: nums = [0,1,0,3,12]
输出: [1,3,12,0,0]
```

解题思路：

核心思想：双指针，一个指针遍历，一个指针记录非零元素应该放置的位置

步骤： 1. 初始化指针j = 0，表示下一个非零元素应该放的位置 2. 遍历数组，指针i从0到n-1： - 如果nums[i] != 0： - 将nums[i]赋值给nums[j] - j++ 3. 将j到数组末尾的所有元素赋值为0

优化版本（一次遍历）： - 遍历时直接交换非零元素到前面 - 保持非零元素的相对顺序

时间复杂度：O(n)
空间复杂度：O(1)

---

6. 盛最多水的容器 (Container With Most Water)

题号：11
难度：中等
链接：https://leetcode.cn/problems/container-with-most-water/

题目描述： 给定一个长度为 n 的整数数组 height 。有 n 条垂线，第 i 条线的两个端点是 (i, 0) 和 (i, height[i]) 。

找出其中的两条线，使得它们与 x 轴共同构成的容器可以容纳最多的水。

返回容器可以储存的最大水量。

说明：你不能倾斜容器。

示例：

```
输入：[1,8,6,2,5,4,8,3,7]
输出：49
解释：图中垂直线代表输入数组 [1,8,6,2,5,4,8,3,7]。在此情况下，容器能够容纳水（表示为蓝色部分）的最大值为 49。
```

解题思路：

核心思想：双指针从两端向中间移动，移动较小高度的指针

为什么移动较小高度的指针？ - 容器面积 = min(height[left], height[right]) × (right - left) - 如果移动较大高度的指针，宽度减小，高度不变或更小，面积一定减小 - 如果移动较小高度的指针，虽然宽度减小，但高度可能增大，面积可能增大

步骤： 1. 初始化left = 0, right = n-1, maxArea = 0 2. 当left < right时： - 计算当前面积：area = min(height[left], height[right]) × (right - left) - 更新maxArea - 如果height[left] < height[right]，left++；否则right–

时间复杂度：O(n)
空间复杂度：O(1)

---

7. 三数之和 (3Sum)

题号：15
难度：中等
链接：https://leetcode.cn/problems/3sum/

题目描述： 给你一个整数数组 nums ，判断是否存在三元组 [nums[i], nums[j], nums[k]] 满足 i != j、i != k 且 j != k ，同时还满足 nums[i] + nums[j] + nums[k] == 0 。

请你返回所有和为 0 且不重复的三元组。

注意：答案中不可以包含重复的三元组。

示例：

```
输入：nums = [-1,0,1,2,-1,-4]
输出：[[-1,-1,2],[-1,0,1]]
```

解题思路：

核心思想：排序 + 双指针

步骤： 1. 对数组进行排序 2. 遍历数组，固定第一个数nums[i]： - 如果nums[i] > 0，break（三数之和不可能为0） - 去重：如果i > 0且nums[i] == nums[i-1]，跳过 - 使用双指针left = i+1, right = n-1： - 计算sum = nums[i] + nums[left] + nums[right] - 如果sum == 0：记录结果，left++，right–，同时去重 - 如果sum < 0：left++ - 如果sum > 0：right–

去重处理： - 外层循环跳过重复的第一个数 - 找到解后，跳过左右指针的重复元素

时间复杂度：$O(n^2)$
空间复杂度：O(1)或O(log n)（排序的栈空间）

---

8. 接雨水 (Trapping Rain Water)

题号：42
难度：困难
链接：https://leetcode.cn/problems/trapping-rain-water/

题目描述： 给定 n 个非负整数表示每个宽度为 1 的柱子的高度图，计算按此排列的柱子，下雨之后能接多少雨水。

示例：

```
输入：height = [0,1,0,2,1,0,1,3,2,1,2,1]
输出：6
解释：上面是由数组 [0,1,0,2,1,0,1,3,2,1,2,1] 表示的高度图，在这种情况下，可以接 6 个单位的雨水（蓝色部分表示雨水）。
```

解题思路：

核心思想：每个位置能接的雨水 = min(左边最大高度, 右边最大高度) - 当前高度

方法一：动态规划 - 预处理每个位置左边的最大高度和右边的最大高度 - 遍历计算每个位置的接水量 - 时间复杂度：O(n)，空间复杂度：O(n)

方法二：双指针（推荐） - 用两个指针从两端向中间移动 - 维护左边的最大值和右边的最大值 - 较小的一侧可以计算接水量（因为接水量由较小的一侧决定）

双指针步骤： 1. 初始化left = 0, right = n-1, leftMax = 0, rightMax = 0, result = 0 2. 当left < right时： - 如果height[left] < height[right]： - 如果height[left] >= leftMax，更新leftMax - 否则，接水量 += leftMax - height[left] - left++ - 否则： - 如果height[right] >= rightMax，更新rightMax - 否则，接水量 += rightMax - height[right] - right–

时间复杂度：O(n)
空间复杂度：O(1)

三、滑动窗口

9. 无重复字符的最长子串 (Longest Substring Without Repeating Characters)

题号：3
难度：中等
链接：https://leetcode.cn/problems/longest-substring-without-repeating-characters/

题目描述： 给定一个字符串 s ，请你找出其中不含有重复字符的 最长子串 的长度。

示例：

```
输入: s = "abcabcbb"
输出: 3
解释: 因为无重复字符的最长子串是 "abc"，所以其长度为 3。
```

解题思路：

核心思想：滑动窗口，维护一个不包含重复字符的窗口

步骤： 1. 初始化left = 0, maxLen = 0 2. 使用哈希集合或哈希表记录窗口中的字符 3. 遍历字符串，right从0到n-1： - 如果s[right]不在窗口中： - 将s[right]加入窗口 - 更新maxLen = max(maxLen, right - left + 1) - 如果s[right]已在窗口中： - 移动left，直到s[right]不在窗口中 - 将s[right]加入窗口

优化：使用哈希表记录字符的索引，可以直接将left跳到重复字符的下一个位置

时间复杂度：O(n)，每个字符最多被访问两次
空间复杂度：O(min(m, n))，m是字符集大小

---

10. 找到字符串中所有字母异位词 (Find All Anagrams in a String)

题号：438
难度：中等
链接：https://leetcode.cn/problems/find-all-anagrams-in-a-string/

题目描述： 给定两个字符串 s 和 p，找到 s 中所有 p 的 异位词 的子串，返回这些子串的起始索引。不考虑答案输出的顺序。

示例：

```
输入: s = "cbaebabacd", p = "abc"
输出: [0,6]
解释:
起始索引等于 0 的子串是 "cba"，它是 "abc" 的异位词。
起始索引等于 6 的子串是 "bac"，它是 "abc" 的异位词。
```

解题思路：

核心思想：滑动窗口 + 字符计数

步骤： 1. 统计p中每个字符的出现次数 2. 维护一个大小为p.length()的滑动窗口 3. 使用数组或哈希表统计窗口中每个字符的出现次数 4. 当窗口中的字符计数与p的字符计数相同时，记录起始索引 5. 滑动窗口：右移一位，移除左边字符，添加右边字符

优化：使用一个变量记录当前窗口与目标差异的字符种类数，避免每次比较整个数组

时间复杂度：O(n)，n是s的长度
空间复杂度：O(1)，固定大小的数组（26个字母）

---

11. 和为 K 的子数组 (Subarray Sum Equals K)

题号：560
难度：中等
链接：https://leetcode.cn/problems/subarray-sum-equals-k/

题目描述： 给你一个整数数组 nums 和一个整数 k ，请你统计并返回 该数组中和为 k 的子数组的个数 。

子数组是数组中元素的连续非空序列。

示例：

```
输入：nums = [1,1,1], k = 2
输出：2
```

解题思路：

核心思想：前缀和 + 哈希表

关键公式： - 子数组和 = prefixSum[j] - prefixSum[i-1] = k - 即 prefixSum[i-1] = prefixSum[j] - k

步骤： 1. 使用哈希表记录每个前缀和出现的次数 2. 遍历数组，计算当前前缀和 3. 在哈希表中查找prefixSum - k出现的次数，加到结果中 4. 将当前前缀和加入哈希表

初始化： - hashMap[0] = 1，表示前缀和为0出现1次（处理从索引0开始的子数组）

时间复杂度：O(n)
空间复杂度：O(n)

四、普通数组

12. 最大子数组和 (Maximum Subarray)

题号：53
难度：中等
链接：https://leetcode.cn/problems/maximum-subarray/

题目描述： 给你一个整数数组 nums ，请你找出一个具有最大和的连续子数组（子数组最少包含一个元素），返回其最大和。

子数组 是数组中的一个连续部分。

示例：

```
输入：nums = [-2,1,-3,4,-1,2,1,-5,4]
输出：6
解释：连续子数组 [4,-1,2,1] 的和最大，为 6 。
```

解题思路：

核心思想：动态规划（Kadane算法）

状态定义： - dp[i]表示以nums[i]结尾的最大子数组和

状态转移方程： - dp[i] = max(nums[i], dp[i-1] + nums[i]) - 要么从当前元素重新开始，要么延续前面的子数组

优化： - 不需要存储整个dp数组，只需维护当前值和最大值

步骤： 1. 初始化currentSum = nums[0], maxSum = nums[0] 2. 从i=1遍历到n-1： - currentSum = max(nums[i], currentSum + nums[i]) - maxSum = max(maxSum, currentSum)

时间复杂度：O(n)
空间复杂度：O(1)

---

13. 合并区间 (Merge Intervals)

题号：56
难度：中等
链接：https://leetcode.cn/problems/merge-intervals/

题目描述： 以数组 intervals 表示若干个区间的集合，其中单个区间为 intervals[i] = [starti, endi] 。请你合并所有重叠的区间，并返回一个不重叠的区间数组，该数组需恰好覆盖输入中的所有区间。

示例：

```
输入：intervals = [[1,3],[2,6],[8,10],[15,18]]
输出：[[1,6],[8,10],[15,18]]
解释：区间 [1,3] 和 [2,6] 重叠, 将它们合并为 [1,6]。
```

解题思路：

核心思想：排序 + 贪心

步骤： 1. 按区间的左端点排序 2. 初始化结果列表，将第一个区间加入 3. 遍历剩余的区间： - 如果当前区间的左端点 <= 结果列表最后一个区间的右端点： - 有重叠，合并：更新最后一个区间的右端点为max(当前右端点, 最后一个右端点) - 否则： - 无重叠，将当前区间加入结果列表

时间复杂度：O(n log n)，主要是排序
空间复杂度：O(log n)或O(n)，取决于排序实现

---

14. 轮转数组 (Rotate Array)

题号：189
难度：中等
链接：https://leetcode.cn/problems/rotate-array/

题目描述： 给定一个整数数组 nums，将数组中的元素向右轮转 k 个位置，其中 k 是非负数。

示例：

```
输入: nums = [1,2,3,4,5,6,7], k = 3
输出: [5,6,7,1,2,3,4]
解释:
向右轮转 1 步: [7,1,2,3,4,5,6]
向右轮转 2 步: [6,7,1,2,3,4,5]
向右轮转 3 步: [5,6,7,1,2,3,4]
```

解题思路：

方法一：使用额外数组（不推荐） - 时间复杂度：O(n) - 空间复杂度：O(n)

方法二：数组反转（推荐） - 思路：先整体反转，再分别反转前后两部分 - 例如：[1,2,3,4,5,6,7], k=3 1. 整体反转：[7,6,5,4,3,2,1] 2. 反转前k个：[5,6,7,4,3,2,1] 3. 反转剩余：[5,6,7,1,2,3,4]

步骤： 1. k = k % n（处理k大于n的情况） 2. 反转整个数组 3. 反转[0, k-1] 4. 反转[k, n-1]

时间复杂度：O(n)
空间复杂度：O(1)

---

15. 除自身以外数组的乘积 (Product of Array Except Self)

题号：238
难度：中等
链接：https://leetcode.cn/problems/product-of-array-except-self/

题目描述： 给你一个整数数组 nums，返回 数组 answer ，其中 answer[i] 等于 nums 中除 nums[i] 之外其余各元素的乘积 。

题目数据 保证 数组 nums之中任意元素的全部前缀元素和后缀的乘积都在 32 位 整数范围内。

请**不要使用除法**，且在 O(n) 时间复杂度内完成此题。

示例：

```
输入: nums = [1,2,3,4]
输出: [24,12,8,6]
```

解题思路：

核心思想：前缀积 × 后缀积

步骤： 1. 创建结果数组answer 2. 第一次遍历（从左到右）：计算前缀积 - answer[i] = nums[0] × nums[1] × … × nums[i-1] 3. 第二次遍历（从右到左）：计算后缀积并乘到结果中 - 维护一个变量suffix，表示从右往左的后缀积 - answer[i] = *suffix* - *suffix* = nums[i]

示例：

```
nums = [1, 2, 3, 4]

第一次遍历后（前缀积）：
answer = [1, 1, 2, 6]

第二次遍历：
i=3: answer[3]=6*1=6, suffix=1*4=4
i=2: answer[2]=2*4=8, suffix=4*3=12
i=1: answer[1]=1*12=12, suffix=12*2=24
i=0: answer[0]=1*24=24, suffix=24*1=24

最终结果：[24, 12, 8, 6]
```

时间复杂度：O(n)
空间复杂度：O(1)，输出数组不算额外空间